A smaller checked presentation of intuitionistic primitive recursive arithmetic

A 673-byte fully validated checker, a 502-byte compact profile, and precise limits of the improvement

Research prepared for Vladimir Reshetnikov

12 November 2026

Abstract

We inspect the current ConciseIPRA collection at commit `6ffc49f55d35` (full identity below), including its three incoming archives. The main deliverable is a standalone proof checker for the existing native Eq/D presentation that validates the entire integer/list certificate alphabet, its closed public goal, and its binding scope. Its normalized size is **673 bytes, 344 significant Python tokens, and 393 AST nodes**, compared with **849 / 370 / 426** for the closest explicitly matched fully typed native control. A separate compact profile reduces the actual byte incumbent from 506 to **502** bytes.

The improvement is in the checked representation and validation architecture; the mathematical axioms and inference rules are inherited. We give their complete specification, explain the whole-domain bi-interpretation with full first-order iPRA, prove the implementation transformations, and include the foundational arithmetic proofs as attributed appendices. The current compact coordinate envelope is 253 tokens, 275 AST nodes, and 506 bytes, attained by different programs. The new results do not beat that envelope in every coordinate and do not establish an absolutely simplest axiomatization. No new Lean or Rocq verification is claimed.

Contents

1 Result, comparison set, and the unresolved part of the request **4**

2 The complete mathematical presentation **4**
- 2.1 Terms, formulas, and freshness . . . 4
- 2.2 All axioms and all primitive rules . . . 5
- 2.3 Why ordinary intuitionistic logic is present . . . 5

3 The bi-interpretation, including both composites **6**

4 What the compact 502-byte profile presents **7**

5 The implementation transformations and their proofs **8**
- 5.1 A single alphabet check discharges repeated type obligations . . . 8
- 5.2 Eager tuple selection of the next proof goal . . . 8
- 5.3 The compact remainder selector . . . 9
- 5.4 A mode distinction that must be preserved . . . 9

6 Measured sizes and executed verification **10**

7 A rejected simplification: the danger of a ternary matrix **11**

8 Reproduction and further research questions **11**

Attributed foundational proofs **13**

A Complementary introductions with one local assumption **13**
A.1 Complete native syntax and rule inventory . . . 13
A.2 Exact equivalence and the unused-witness construction . . . 14
A.3 Induction and numerical cases separate cleanly . . . 15
A.4 The entire ordinary logical interface . . . 16
A.5 Whole-domain realizations and both formula composites . . . 17
A.6 What the theorem does and does not claim . . . 18

B A guarded core with a deterministic introduction rule **18**
B.1 Complete syntax, axioms, and inference rules . . . 18
B.2 Padding, equation implication, and numerical cases . . . 20
B.3 Recovery of contextual equation induction . . . 21
B.4 Exact translations to and from the inherited calculus . . . 21
B.5 Why an unrestricted checker can still be sound . . . 23
B.6 Why merely deleting the choice would not suffice . . . 23

C Internal scalar predecessor bootstrap **24**
C.1 Elementary terms and two finite cycles . . . 24
C.2 A ramp with a global scaling identity . . . 25
C.3 A comparator from scalar iteration . . . 25
C.4 Saturated iteration gives predecessor . . . 26
C.5 Dependency boundary . . . 27

D From literal-equation induction to full intuitionistic PRA **27**
D.1 The exact hypotheses . . . 28
D.2 Constructor separation and two elementary tests . . . 28
D.3 Four elementary uses of equation induction . . . 29
D.4 An equation invariant that reflects actual equality . . . 30
D.5 Compilation of all quantifier-free formulas . . . 31
D.6 Recovery of induction and the full theory . . . 32

E Pairing, primitive recursion, and internal numerical composites **33**
E.1 Pairing from a bounded square-root search . . . 33
E.2 Compiling each primitive recursion . . . 34
E.3 Why both numerical round trips are internal . . . 34

F Arithmetic strength and both ordinary interpretation composites **34**
F.1 The arithmetic dependency order . . . 35

G Removing literal zero from the source of transport **36**
G.1 Structural operations, cases, and equation induction . . . 37
G.2 The equality operations available immediately . . . 37
G.3 A singleton-one test without predecessor . . . 38
G.4 Turning a zero-source equality around . . . 39

G.5 Recovering equality algebra and formula transport . . . 39
G.6 Compatibility with restricted computation certificates . . . 40
G.7 Scope of the result . . . 41

H A uniform arity encoding and its checking theorem 41
H.1 The complete sorted interpretation of arrays . . . 41
H.2 One depth calculation replaces the lookup table . . . 42
H.3 Opening and scope . . . 42
H.4 A computation hypothesis only needs a zero instance . . . 43
H.5 Audit of the logical continuations . . . 44
H.6 Canonical completeness, including zero witnesses . . . 45
H.7 JSON and finite-runtime qualifications . . . 46

I Complete selected source listings 48
I.1 Fully validated native checker . . . 48
I.2 Compact guarded checker . . . 48

1 Result, comparison set, and the unresolved part of the request

The requested criterion is the size of a complete proof checker, while the theory must remain bi-interpretable and equiconsistent with full first-order intuitionistic primitive recursive arithmetic. This report gives a verified improvement for a precisely specified complete-checker contract. It does not claim a new arithmetic basis or a new deletion of a mathematical axiom.

The repository distinguishes several interfaces that must be kept separate:

1. A *compact* program assumes a canonical guarded public goal and an integer/list certificate alphabet. It need not enforce both assumptions itself.
2. A *goal-validated* program checks its public formula and scope, while allowing the inherited treatment of auxiliary data and unused prefixes.
3. An *entire-certificate-validated* program checks every supplied leaf and container, including unused prefixes, as well as the public formula and its scope. The selected native program has this contract.

All reported module sizes include the JSON reader, the checker itself, and the final acceptance output. None of the selected checkers imports a local helper, an external theorem prover, a compressed hidden checker, or a table of theorems. Python and its standard JSON library are shared infrastructure, as in the repository's measurement convention.

The current source comparison was pinned to the following full commit:

`6ffc49f55d355b548f7a8e1530570aee5de086b3`.

The audit matched all 5,825 Python paths in the ninety Research packages against the frozen corpus, with zero added, missing, or changed paths, and rechecked 1,380 distinct Git blobs and their normalized measurements. The incoming `ipra_smaller_envelope.zip` improves the token and AST coordinates of that Research corpus. Thus the relevant compact envelope is

$$(253 \text{ tokens},\ 275 \text{ AST nodes},\ 506 \text{ bytes}).$$

It is an observed coordinatewise minimum over a classified collection of programs, not a proved lower bound on all presentations of arithmetic. The three coordinates do not belong to one checker.

The main strict comparison is with `Research/validated_presentation/code/native_typed_eqd.py`, whose Git blob is `698c76daf322529ae4b321d67097c7b99c95874d`. Its native Eq/D goal grammar and whole-certificate alphabet contract are the relevant match. Smaller 656-, 777-, and 778-byte native programs in the collection validate their goal but do not impose the same whole-certificate alphabet gate. An 820-byte typed program accepts the richer Eq/Q/D public language; it is recorded separately in the comparison ledger.

Precise status. There is a smaller fully validated implementation of the existing axiomatization, and a smaller compact byte profile. There is no established new mathematical axiomatization below every coordinate of the current compact envelope. In particular, the phrase "simplest axiomatization" cannot be promoted here to a proved global minimum.

2 The complete mathematical presentation

2.1 Terms, formulas, and freshness

There is one numerical sort. Terms are fixed finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in h is bound by an iterator. Its update may contain any fixed numerical parameters, but has no extra recursion-counter argument. Every body is finite earlier syntax. Equivalently, elaborate each fixed body into its own first-order function symbol with parameters. This convention does not assert a total numerical universal evaluator for arbitrary primitive recursive programs.

The native formulas are

$$A ::= (s = t) \mid \mathsf{Q}_y B \mid \mathsf{D}_{x,y} B.$$

The public language of the selected checker omits Q; closures are available internally. The designated contradiction is $0 = S0$. Contexts Γ are finite lists of reusable native assumptions. The variables x, y bound by a paired binder are distinct. All displayed substitution is capture avoiding. Bound variables are renamed before substitution when necessary.

2.2 All axioms and all primitive rules

The two nonlogical axiom schemas, available in any context, are

$$I(0, a; u.h) = a, \tag{C0}$$

$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{CS}$$

Together with these schemas, the following six rule families specify the presentation. The two introduction rows are distinct allowed patterns.

$$\frac{A \in \Gamma}{\Gamma \vdash A} \quad \text{Hyp},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad \text{Tr},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \quad \text{QE},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A} \quad \text{DE},$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)} \quad \text{DI}_{\text{case}},$$

$$\frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x, y)} \quad \text{DI}_{\text{progress}}.$$

Here F is any native formula template, and E is a *literal equation*. An equation beneath a binder is not a literal equation. In DE, b is fresh for the conclusion, context, and passive parameters of the matrix. In either introduction, z is fresh for the context, witness t, and passive parameters. The displayed local assumptions are discharged.

This is the inherited Choice-DI calculus. Counting each computation schema as a rule gives eight patterns. There are no additional primitive ordinary logical axioms, successor axioms, equality-reflexivity axiom, or induction schemas. The proof checker uses an equivalent packet protocol: some of the displayed rules are recovered as derived rules rather than assigned distinct instruction tags.

2.3 Why ordinary intuitionistic logic is present

Put $C(v, a, b) = I(v, a; u.b)$, with u unused in b. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define a gate recursively on native Eq/D formulas by

$$G_v(s = t) \equiv C(v, 0, s) = C(v, 0, t), \qquad G_v(\mathsf{D}_{x,y} B) \equiv \mathsf{D}_{x,y} G_v B.$$

The calculus proves $G_0 A$, and $G_{Sv} A$ is interderivable with A. It also proves the padding equivalence $A \dashv\vdash \mathsf{D}_{u,y} G_y A$, with u unused.

For formulas of equal binder shape, select between their final equations using C, and commute this operation through aligned D binders. For unequal shapes, pad the shallower equation with the preceding gate. This defines $M_v(A, B)$ with endpoint equivalences $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$. The full recursive definition and derivations are in Appendix A; no propositional connective is silently assumed by that construction.

Let $\delta(v) = C(v, S0, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. The ordinary logical operations are the following macros, with fresh displayed binders and unused u:

$$\begin{aligned} A \Rightarrow_* B &= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &= (0 = S0). \end{aligned}$$

They admit the ordinary intuitionistic introduction and elimination rules. Numerical cases are derived by case DI with an unused witness coordinate, followed by DE and QE. Equation induction is obtained similarly from progress DI. Explosion follows because $0 = S0$, computation, and transport make $s = C(0, s, t) = C(S0, s, t) = t$ for arbitrary terms; induction on formula syntax then yields every native formula. No excluded middle for arbitrary formulas is introduced.

3 The bi-interpretation, including both composites

The ordinary first-order realization T_I uses intuitionistic equality logic on the same numerical domain, the computation equations C0 and CS, contextual induction for literal equations, and the two explicit axioms

$$\forall n\, (n = 0 \vee \exists m\; n = Sm), \qquad (0 = S0) \to \bot.$$

The second is an actual falsity bridge. It cannot be omitted on the ground that the native presentation derives explosion from its designated equation: positive computation equations alone admit a singleton structure.

The interpretations of the two native binders are

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ(y), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x (B^\circ(x, 0) \to \forall y B^\circ(x, y)).$$

Equation, variable, and numerical term interpretation is the identity. Case DI is sound by numerical coverage. Progress DI is sound by equation induction under the temporarily assumed zero instance. The other displayed rules have their ordinary equality and logical meanings. Conversely, the macros and derived rules above interpret T_I in the native calculus.

Theorem 3.1 (Inherited whole-domain arithmetic bridge)**.** *The ordinary realization T_I, and consequently the native presentation, is bi-interpretable and equiconsistent with full first-order* iPRA*. Both numerical interpretation domains are the whole domain, interpreted equality is actual equality, and both numerical composite isomorphisms have graph $x = y$.*

Proof. The iPRA interpretation of a fixed iterator uses primitive recursion with an update that ignores its counter. It proves C0, CS, coverage, separation, and every contextual equation-induction instance.

The reverse direction uses the following internal constructions in order. First the scalar predecessor proof gives a fixed term P satisfying $P0 = 0$ and $P(Sn) = n$. This yields successor injectivity, and coverage strengthens to $n = 0 \vee n = S(Pn)$. Next, truncated subtraction and the equality-reflection construction give a term that detects actual equality on the entire domain. Each fixed quantifier-free formula consequently has a primitive recursive Boolean characteristic, so literal-equation induction yields quantifier-free induction. Only after those steps are pairing and its projections constructed and verified. The proofs are reproduced in Appendices C and E; the equality-reflection appendix lies between them. This order avoids assuming equality detection or pairing in order to obtain the predecessor.

If π, L, H are pairing and projections, a fixed primitive recursion $f(0, \vec{p}) = g(\vec{p})$, $f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$ is compiled as

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))),$$
$$f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the projection identities then prove the defining equations of f^*. This is external recursion on each fixed finite function description, not object-level universal evaluation of function codes.

For the numerical composites, a double-translated iterator has the same initializer and the same update after its lower-level descriptions have been identified. Equation induction on equality of two recurrence values proves their uniqueness. External induction on finite descriptions thus identifies every operation with its double translation. The same argument works for counter-aware primitive recursion. Whole domain and actual equality make the graph $x = y$ total, bijective, and operation preserving.

For the formula composites, the ordinary readings of the six logical macros are intuitionistically equivalent to their intended operations. The essential native composite is

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y B(x, y)\big).$$

Forwards, open D, assume its base, and use its closure at an arbitrary fresh parameter before introducing the macros. Backwards, eliminate the existential at a fresh witness and use case DI, obtaining the successor instance from the implication and universal. Structural induction gives both formula round trips. The falsity bridge is preserved, and effective translations of finite contradiction proofs give equiconsistency. □

This theorem is inherited mathematics, with complete attributed supporting proofs included below. The new contribution is the checked implementation comparison. The repository contains Lean proofs of many native-calculus equivalences; this report neither rebuilds them nor claims that the new Python modules have been verified by Lean.

4 What the compact 502-byte profile presents

The compact profile uses the same scalar terms and a guarded Eq/D public grammar. An equation immediately beneath $\mathsf{D}_{x,y}$ must have the literal shape

$$I(y, 0; u.h) = I(y, 0; u.k).$$

Its zero instance is provable by computation. Compound matrices use case introduction; literal equations use progress introduction. Arbitrary native formulas are translated by guarding and

padding their atomic parts. The padding $\mathsf{D}_{u,y}G_yE$ is equivalent to an equation E, so this restriction does not lose the represented first-order theory. Appendix B proves the guarded/native equivalence.

The source-selected transport protocol reserves literal source zero for closure elimination. Its missing zero-source transport instances are derived; the corresponding proof is included in the zero-source appendix. Thus a short packet format should not be mistaken for a rule that simply forbids an unrecoverable use of equality.

The 502-byte file assumes the canonical guarded public contract. The optional exact guarded wrapper includes the validator and therefore has its own measured size, 1,172 bytes. It is not the preferred standalone interface: the selected native checker validates all its input in 673 bytes and exposes the more convenient native Eq/D language directly. The compact profile is included because it improves the actual compact byte record, not because its missing validation is free.

5 The implementation transformations and their proofs

5.1 A single alphabet check discharges repeated type obligations

After JSON parsing, the selected native checker uses the gate

```
q = not str(p).strip('0123456789-[], ') and p.pop
```

and then calls the resulting stack operation. The gate and the driver together require a top-level list whose descendants are only integers and lists. In the Python representation of such a tree, every character belongs to the displayed alphabet. Conversely, a Boolean or null uses letters, a float uses a decimal point or a nonnumeric spelling, a nested string has quote characters, and an object has braces and a colon. At least one forbidden character remains after stripping allowed characters from the ends. Top-level strings such as a string of digits are a necessary exception to the alphabet argument by itself: even if that predicate passes, they have no `pop` method and are rejected by the driver before proof checking. Top-level integers likewise cannot enter the checker.

For every finite string s and character alphabet A, $s.\mathrm{strip}(A)$ is empty exactly when all characters of s are in A. Thus this is an exact alphabet test, not a heuristic test of the first and last characters.

During the initial public-goal traversal, the old additional `type(a)isint` check is consequently redundant. If the list branch does not apply, an actual list reaches an invalid scalar comparison and rejects; every successful scalar case is already an integer. Later traversals had the initial-validation phase disabled, so removing that test changes no later rule. For all reachable integer/list data, and any internally generated Boolean zero, the tests `a*0==[]` and `a*0!=0` select the same list branch. The latter saves a further token, AST node, and byte. The missing-image function sentinel still rejects.

The remaining initial traversal checks the closed Eq/D grammar, successor/iterator term grammar, and binding scope. An internal phase bit enables these public checks only at the first call; the JSON certificate cannot set it. This safely shares grammar traversal with the scope traversal already required by the checker.

5.2 Eager tuple selection of the next proof goal

The incumbent next-goal expression has the shape

$$C \text{ and } R \text{ or } a,$$

where, after computing the newly retained hypothesis,

$$C \equiv (m, a) = (s \text{ and } [s, u] \text{ or } [t, t, t],\ i(t, u)),$$
$$R \equiv i(t, s).$$

The new expression is `(a,i(t,s))[C]`. The opener i is pure on finite integer/list trees; the caller's scope floor is local and is not changed by the recursive proof of m. Both openings occur before the same floor decrement.

Lemma 5.1 (False-result lemma)**.** *At formula-root depth zero, if both $i(t, s)$ and $i(t, u)$ terminate and the former is false in Python, they are equal.*

Proof. For a list t, the opener preserves the outer list length. A false result therefore means t was empty, and both results are empty. For an integer t, the root-depth-zero branch is independent of the replacement image: it either retains a scoped negative parameter or returns zero. A false result is zero in both calls. □

If C is false, both selectors return a whenever the eager opening terminates. If C is true and R is true, both return R. If C is true and R is false, the lemma and $a = i(t, u)$ give $a = R$, so the choices again agree. Every successful eager execution therefore has the old selected goal. The new computation can reject a noncanonical certificate that the old short-circuit expression never inspected. It does not follow that the raw accepted certificate sets are equal.

Canonical completeness is preserved: numerical fields use only odd list arities, so a formula-root traversal of such a field stays at even depth and does not inspect its image. Genuine elimination packets already needed the selected opening. Inert fields can be chosen to be zero. Hence canonical proofs have the required extra terminating opening.

5.3 The compact remainder selector

Only the compact profile changes `~len(a) % 2` to `3 % len(a)`. The meaningful formula arities are one for D, two for equality, and three for Q. The two expressions agree there:

r	1	2	3
$(\sim r) \bmod 2$	0	1	0
$3 \bmod r$	0	1	0

At zero arity the new expression rejects. At arities at least four the inherited sorted decoder interprets the entire formula as administrative truth. Changing the opening image there cannot add a nontrivial assumption. For a generated computation, the relevant inherited justification is the known-zero-instance supply lemma: substitute newly admitted names to zero and cut in the resulting computation theorem. It is not necessary, and in general not correct, to claim the fresh-parameter supply is uniformly a theorem.

This is a semantic soundness argument. The composed compact program is not claimed to have raw accepted-trace inclusion in the 506-byte control at all surplus arities. Its claim is the same canonical public theorem set. The selected fully typed native program retains its original introduction-mode test unchanged.

5.4 A mode distinction that must be preserved

In the native packet protocol, both the integer 0 and the empty list [] are false as control flags, but they are unequal. For an equation matrix, the condition `len(a) % 2 == u` selects progress at $u = 0$,

and cases at $u = []$. Both native introduction patterns are therefore available. Replacing these two falsy values by one bit would silently remove a required case. The exact alphabet gate admits both values and the new native code preserves the test.

Theorem 5.2 (Implementation claim)**.** *Under the specified JSON protocol, the selected native module validates the entire integer/list alphabet and its closed native Eq/*D *public goal. On the decoded public language it is sound and canonically complete for the inherited native presentation, and hence presents a theory bi-interpretable and equiconsistent with* iPRA*.*

Proof. The gate and initial traversal give the input invariants above. The type-check removal and the integer/list discriminator replacement are exact under those invariants. The tuple-selector lemma gives soundness relative to the inherited native checker, and the canonical-packet argument preserves theorem completeness. The remaining source rewrites are the exact Boolean selection of a nonempty pair and the integer identity $\sim(-k) = k - 1$. The native checking theorem and Theorem 3.1 then compose. The raw sorted decoder, scope invariant, and known-zero-instance lemma are reproduced in Appendix H. □

6 Measured sizes and executed verification

The measurements use CPython 3.12.14. Source is parsed, normalized by `ast.unparse`, and given one final line feed. Significant tokens exclude encoding, layout, end marker, and comments. AST counts include context and operator nodes. The measurement scripts and frozen controls are included, so the figures can be recomputed directly.

Complete module and contract	Tokens	AST	Bytes
Matched fully typed native Eq/D	370	426	849
New fully typed native Eq/D	**344**	**393**	**673**
Goal-validated native control	333	379	656
Eager goal-validated native intermediate	338	385	647
Compact guarded byte incumbent	267	303	506
New compact guarded profile	269	301	502
Matched exact guarded wrapper	461	521	1176
New exact guarded wrapper	463	519	1172

The selected fully typed native module improves all three coordinates against the matched 849-byte control. Against the separate compact envelope, only the compact byte coordinate improves: it becomes 253/275/502, still attained by different programs. The selected native module does not beat 253 tokens, 275 AST nodes, or 502 bytes. Adding its validation obligations to an unvalidated compact source is not a zero-cost operation.

The compact suite executed 292 standalone subprocess checks, including ordinary and optimized Python: 142 expected acceptances, 150 expected rejections, and zero discrepancies. The native suite's actual run executed 145 standalone checks with zero discrepancies; the complete per-case receipts accompany the source. These suites cover computation, transport, a used induction hypothesis, compound logical introduction and elimination, both atomic introduction modes, malformed public syntax, scope escape, context leakage, false goals, invalid auxiliary data and prefixes, and the intentional eager-only rejection.

All acceptance-critical checks remain active under `python-O`. The acceptance convention is exit status zero and exactly `ACCEPT` followed by a line feed. Exceptions reject. Unused prefixes are allowed

as in the inherited protocol, but the selected native program still checks their alphabet. Finite Python stack, memory, and integer conversion limits can reject large certificates; the mathematical completeness statement uses unbounded finite resources, as do the controls.

These executions are implementation evidence. They do not replace the general proofs, prove consistency by testing, or constitute proof-assistant verification of the Python checker.

7 A rejected simplification: the danger of a ternary matrix

One tempting attempted refinement was to choose the opening image solely from the newly retained hypothesis h, using `(len(h)>1)andk`. It handles an equation and an exported closure uniformly and appears to remove the explicit major-premise flag. It fails at a meaningful ternary matrix: an introduction whose matrix is a Q formula would receive the progress mode. Public Eq/D validation does not forbid Q inside auxiliary proofs of major premises.

Proposition 7.1 (Closure progress is not an innocent mode extension)**.** *Adding progress introduction for arbitrary* Q*-rooted matrices to the native calculus supplies contextual induction for arbitrary native formulas, rather than just literal equations.*

Proof. For any native formula $A(n)$, extend the gate by $G_v(\mathsf{Q}_y B) = \mathsf{Q}_y G_v B$, renaming y fresh for v, and put $F(n) = \mathsf{Q}_v G_v A(n)$. Gate-zero is provable, and gate removal at 1 gives $F(n) \dashv\vdash A(n)$. Conversely, under $A(n)$, numerical cases give every gate instance and hence the closure. These conversions use the ordinary native rules, not induction on arbitrary formulas.

Given a base proof of $A(0)$ and a step from $A(z)$ to $A(Sz)$, convert them to the corresponding base and step for F. The proposed extra progress introduction, with an unused witness coordinate, gives $\mathsf{D}_{x,n}F(n)$. DE opens its closure; the base and QE give $F(t)$, then $A(t)$. Thus the intended induction boundary has disappeared. □

This rejected branch is included to make the crucial mathematical constraint reviewable. The delivered programs do not use it. The independent reviews in the archive record this rejection as well as the approved transformations.

8 Reproduction and further research questions

From the extracted archive, run

```
python3 reproduce.py
python3 build_pdf.py
```

The first command recomputes the source metrics and reruns the substantive native and compact certificate suites. The second compiles this report with pdfLaTeX and no shell escape. All mathematical TeX inputs are included. The source-level comparison ledger records the exact repository paths and Git blob identities; the final manifest records delivered file hashes. No repository was modified by this research task.

The remaining questions are mathematical, not requests to accept another unproved size claim:

1. Can an arithmetic or logical rule be genuinely removed while giving a smaller complete checker under one fixed, fully validated public contract? The present reductions change validation and certificate selection, not the axiom inventory.

2. Does successor-only iteration uniqueness imply an actual-equality discriminator in the scalar language? The current orbit-closure report derives zero-equation induction, but full equality

reflection and hence the full iPRA bridge remain unproved for that restriction. Its stronger arbitrary-update uniqueness result must not be reused without proof.

3. Can the remaining induction-independence question be resolved with a model of the binding iterator that respects substitution and passive parameter coherence? Models with independently chosen recursion symbols do not automatically supply such a model.

4. Can the Python acceptance relation, including its totalized arity decoder and scope transitions, be linked to the existing Lean calculus by a verified compiler? This would replace the current written implementation metatheorems with a checkable formal bridge.

5. Which lower bounds can be proved after fixing the programming language, input protocol, allowed libraries, and formula interface? A finite repository census alone cannot prove global optimality.

Attributed foundational proofs

The following sections are retained foundational source fragments from the current ProveIt collection, reproduced under its MIT No Attribution license. They are included to make the arithmetic and logical bridge inspectable inside this deliverable. Historical wording such as "new" in these fragments refers to their original report, not to a novelty claim by the present report. Their original notation and labels are preserved. The selected implementation uses a caller-local floor; any historical discussion of a global floor is the inherited comparison protocol, with freshness preserved by the local-scope refinement.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the

progress choice. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0,t;u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction

uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases.

The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B A guarded core with a deterministic introduction rule

The change is to the public formula language and the introduction rule, not to the numerical domain. An atomic matrix immediately below a paired binder must have a fixed, syntactically recognizable zero guard. Such a matrix has a provable zero instance. Consequently it needs only the progress introduction; the separate atomic case introduction can be deleted. Compound matrices retain the case introduction.

B.1 Complete syntax, axioms, and inference rules

Numerical terms are those of the inherited scalar calculus:

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the displayed variable u is bound in h. There is one numerical sort, fixed finite term syntax, and capture-free substitution. Put

$$C(v, a, b) := I(v, a; u.b), \qquad 1 := S0,$$

where u is fresh. For a literal equation $E \equiv (s = t)$, define

$$G_v E \;:=\; \big(C(v, 0, s) = C(v, 0, t)\big). \tag{Guard}$$

The displayed abbreviations expand into terms of the given syntax.

Call a literal equation a *zero-initialized matrix in y* when it has the syntactic form

$$B(x,y) \equiv I(y,0;u.h) = I(y,0;u.k). \qquad \text{(Canonical atomic matrix)}$$

Here h, k may depend arbitrarily on the state variable u, on x, y, and on fixed parameters; state binders are renamed as necessary. The selector guard $G_y E$ is a special case, with state-independent updates. Every matrix of (Canonical atomic matrix) has the theorem $B(x,0)$ by the two zero computations and equality.

The canonical formulas of $\mathcal{G}$ are generated as follows:

1. Every literal equation $s = t$ is a formula.
2. $\mathsf{D}_{x,y}B$ is a formula when B is already a compound canonical formula (its root is D).
3. $\mathsf{D}_{x,y}B(x,y)$ is a formula whenever B has the zero-initialized shape (Canonical atomic matrix).

The bound variables x, y are distinct. Rule 3 is the restriction on atomic matrices. It does not require an equality decision procedure or a proof that an arbitrary base is true: the guard has the displayed literal shape. The designated contradiction is $0 = 1$.

Retained contexts contain reusable clauses $\langle A; y.B\rangle$, where A, B are canonical formulas and $y \notin \mathrm{FV}(A)$. An ordinary assumption F abbreviates the constant clause $\langle H; v.F\rangle$, where v is fresh and H is the fixed computation theorem $I(0,0;u.0) = 0$.

The two nonlogical axiom schemas, available in every context, are

$$I(0,a;u.h) = a, \qquad I(Sn,a;u.h) = h[I(n,a;u.h)/u]. \qquad \text{(I0,IS)}$$

Besides these, there are precisely the following rules:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \text{if } \langle A; y.B(y)\rangle \in \Gamma \qquad \text{(Use)},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \text{(Tr)},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F} \qquad \text{(DE)},$$

and the single, root-determined introduction schema

$$\frac{\Gamma, B(t,\eta_B(z)) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases} \qquad (\mathrm{DI}_{\mathrm{root}})$$

There is no certificate choice between these two cases. At an atomic matrix the guard condition on the conclusion is mandatory in the canonical calculus. In DE, a is fresh for Γ, F and the fixed parameters. In DI, z is fresh for Γ, t and the fixed parameters. The displayed local resources are discharged. Transport ranges over canonical formula templates, and capture-free numerical substitution preserves their guard shape.

There are no additional primitive logical rules, reflexivity axioms, successor axioms, coverage axioms, or induction schemas. Weakening, exchange, contraction, renaming, numerical substitution, and ordinary-assumption cut are admissible by induction on derivations. For cut, replace a use of the corresponding constant clause by the supplied proof, renaming eigenparameters first.

B.2 Padding, equation implication, and numerical cases

Write $A \dashv\vdash B$ for interderivability under arbitrary additional retained context. The following preliminary arguments use no contextual induction rule.

Lemma B.1 (Equality and guard endpoints)**.** *Reflexivity, symmetry, transitivity, numerical congruence, and their usual contextual uses are derivable. Moreover,*

$$\vdash G_0 E, \qquad G_{Sz} E \dashv\vdash E.$$

Proof. For reflexivity, put $r = I(0, t; u.0)$. The computation equation $r = t$ serves as both premises of transport with motive $v = t$, yielding $t = t$. Standard equality motives then give the remaining equality rules. The two guard claims follow from the two computation equations for C, together with equality transport. □

For an equation E let

$$L(E) := \mathsf{D}_{u,v} G_v E, \qquad P(A) := \begin{cases} L(A), & A \text{ is an equation,} \\ A, & A \text{ is compound.} \end{cases} \tag{Padding}$$

The newly bound u, v are fresh for E. In particular, $P(A)$ is always compound.

Lemma B.2 (Padding)**.** *$L(E) \dashv\vdash E$, and therefore $P(A) \dashv\vdash A$.*

Proof. Under E, atomic progress introduction derives $L(E)$: its target $G_{Sz}E$ follows from the retained E, independently of the local induction assumption. Conversely, DE on $L(E)$ opens $\langle G_0 E; v.G_v E\rangle$. Supply the guard-zero theorem and use the clause at 1; guard removal gives E. □

We will need to reason by numerical cases under an assumption that contains the case variable. It is essential not to violate the eigenvariable condition by directly retaining that assumption during case introduction. A small implication macro resolves this issue.

For equations $U \equiv (s = t)$ and $V \equiv (r = q)$ put

$$M_w(U, V) := (C(w, s, r) = C(w, t, q)), \qquad U \Rightarrow_g V := \mathsf{D}_{a,w} L(M_w(U, V)), \tag{Equation implication}$$

with fresh padding variables and unused a. The matrix of the outer binder is compound.

Lemma B.3 (Implication between equations)**.** *The macros in* (Equation implication) *admit both rules*

$$\frac{\Gamma, U \vdash V}{\Gamma \vdash U \Rightarrow_g V}, \qquad \frac{\Gamma \vdash U \Rightarrow_g V \qquad \Gamma \vdash U}{\Gamma \vdash V}.$$

Proof. The endpoint identities for C and padding give $L(M_0(U, V)) \dashv\vdash U$ and $L(M_{Sz}(U, V)) \dashv\vdash V$. Compound case DI therefore gives introduction. DE, the first endpoint, Use at 1, and the second endpoint give elimination. These endpoint conversions use numerical transport with parameters outside the fresh padding binders; they do not assume a binder-congruence theorem. □

Lemma B.4 (Numerical cases)**.** *For every canonical $A(y)$ the following rule is admissible:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}, \qquad z \text{ fresh for } \Gamma \text{ and the fixed parameters.} \tag{NC}$$

Proof. First suppose A is compound. Case DI with an unused witness coordinate introduces $\mathsf{D}_{u,y} A(y)$, ignoring its local base assumption in the successor premise. DE opens $\langle A(0); y.A(y)\rangle$; supply the given base proof and use it at t.

If A is an equation, apply this already proved compound case rule to $L(A(y))$. Padding converts the two premises to its endpoints and converts the conclusion back. No progress induction is being used in this construction beyond the special padding introductions. □

B.3 Recovery of contextual equation induction

Theorem B.5 (Equation induction)**.** *For every literal equation $E(y)$, $\mathcal{G}$ derives*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad z \text{ fresh for } \Gamma \text{ and the fixed parameters.} \tag{EI}$$

Proof. Put $J(y) = G_y E(y)$. We first prove the progress premise $\Gamma, J(z) \vdash J(Sz)$ without making an invalid contextual use of NC. Consider the compound formula

$$\Phi(y) := J(y) \Rightarrow_g E(Sy).$$

Under Γ, prove $\Phi(0)$ by equation-implication introduction: ignore its local hypothesis, use the supplied proof of $E(0)$, and apply the supplied step proof specialized at 0 to obtain $E(1)$.

For fresh w, prove $\Phi(Sw)$ by equation-implication introduction. Its local hypothesis $J(Sw)$ gives $E(Sw)$ by guard removal, and the supplied step proof specialized at Sw gives $E(SSw)$. These two branch proofs have retained context Γ, independent of the case variable. Thus NC legitimately gives $\Phi(z)$ under Γ. Under the local hypothesis $J(z)$, equation-implication elimination gives $E(Sz)$, hence $J(Sz)$ by the successor guard endpoint.

Atomic progress DI now introduces $\mathsf{D}_{u,y} J(y)$ with unused u. DE and the theorem $J(0)$ yield $J(Sw)$, and hence $E(Sw)$, for a fresh w. Finally NC for E uses the supplied base $E(0)$ and these successor instances to conclude $E(t)$.

The only induction-like premise used here is the single displayed guarded progress introduction. In particular, no predecessor, coverage disjunction, formula induction, or pre-existing EI is assumed. □

B.4 Exact translations to and from the inherited calculus

Let $\mathcal{C}$ be the inherited Choice-DI calculus: the same scalar terms, arbitrary formulas $s = t$ and $\mathsf{D}_{x,y} B$, the same computation, Use, DE, and transport rules, with case DI at every matrix and progress DI at literal equations. Define $T : \mathcal{C} \to \mathcal{G}$ on finite formulas by

$$T(s = t) = (s = t), \qquad T(\mathsf{D}_{x,y} B) = \mathsf{D}_{x,y} P(TB). \tag{Translation}$$

Each padding binder uses fresh names. This is a computable structural translation, preserves free numerical variables, commutes with capture-free numerical substitution up to bound-variable renaming, and translates every paired matrix to a compound matrix. Translate clauses by $T\langle A; y.B\rangle = \langle TA; y.TB\rangle$. Let $J : \mathcal{G} \to \mathcal{C}$ be literal inclusion.

Theorem B.6 (Translation of derivations)**.**

$$\Gamma \vdash_{\mathcal{C}} A \Longrightarrow T\Gamma \vdash_{\mathcal{G}} TA, \qquad \Gamma \vdash_{\mathcal{G}} A \Longrightarrow J\Gamma \vdash_{\mathcal{C}} JA.$$

The transformations are effective on finite derivations.

Proof. Inclusion is immediate: every guarded rule is an allowed Choice-DI rule. For T, induct on a Choice-DI derivation.

Computation is unchanged. Use commutes with substitution, and transport uses the translated motive. A case DI premise translates to a proof from $TB(t, 0)$ to $TB(t, Sz)$. Under the local assumption $P(TB)(t, 0)$, unpad, apply that translated proof, and pad the result. Compound case DI then proves the required $\mathsf{D}_{x,y} P(TB)$.

For a progress DI with equation matrix E, its translated step is still $\Gamma, E(t, z) \vdash E(t, Sz)$. To introduce $\mathsf{D}_{x,y} L(E(x, y))$, use compound case DI. Its retained base $L(E(t, 0))$ supplies $E(t, 0)$. The

recovered contextual EI, under that base, gives $E(t, Sz)$, which padding converts to the target $L(E(t, Sz))$.

For DE, the translated major is $\mathsf{D}_{x,y}P(TB)$. It opens the clause $\langle P(TB)(a,0); y.P(TB)(a,y)\rangle$. The recursively translated minor expects $\langle TB(a,0); y.TB(a,y)\rangle$. Replace every use of this expected clause by: pad its antecedent, use the opened clause at the same term, and unpad its consequent. This effective local proof replacement preserves the fresh witness condition and the conclusion. □

Lemma B.7 (Compound-matrix congruence)**.** *If canonical compound matrices B, C are uniformly interderivable, then $\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}C$ in $\mathcal{G}$.*

Proof. Open the source binder at a fresh witness a. Introduce the target by compound case DI at that witness. Convert its retained base to the source base, use the opened source clause at Sz, and convert the successor instance to the target instance. Close DE. The reverse conversion proves the converse. □

Lemma B.8 (The guarded atomic composite)**.** *For an admissible zero-initialized atomic matrix $B(x, y)$,*

$$\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}L(B) \quad \textit{in } \mathcal{G}. \tag{Atomic composite}$$

Proof. Forwards, open the source clause at a. Introduce the target by compound case DI: unpad its base, use the source clause at Sz, and pad the result.

Backwards, open the clause of $\mathsf{D}_{x,y}L(B)$ at a. The two zero computations prove $B(a,0)$, and padding proves $L(B(a,0))$. Thus the opened clause gives $L(B(a,Sz))$, and unpadding gives $B(a,Sz)$, independently of the local assumption $B(a,z)$. Atomic progress DI therefore introduces $\mathsf{D}_{x,y}B$ at witness a. Close DE. This argument uses the true base explicitly; it would fail for an arbitrary unguarded atomic matrix. □

Theorem B.9 (Both composites and bi-interpretability)**.** *For all $\mathcal{C}$-formulas A and all canonical $\mathcal{G}$-formulas B,*

$$A \dashv\vdash_{\mathcal{C}} JT(A), \qquad B \dashv\vdash_{\mathcal{G}} TJ(B).$$

Consequently $\mathcal{C}$ and $\mathcal{G}$ admit effective structural intertranslations preserving their full consequence relations, with both native formula composites and literal identity numerical translations.

Proof. In $\mathcal{C}$, padding is interderivable with its argument, and case introduction proves binder congruence for arbitrary matrices. Structural induction gives the first displayed equivalence.

For the second, induct on a canonical B. Equations are unchanged. At a compound matrix, the induction hypothesis and compound-matrix congruence apply. At a guarded atomic matrix, the statement is exactly (Atomic composite). The equivalences are uniform under retained assumptions, so they extend to clauses and sequent contexts by the same local replacement of Use instances.

Both interpretations leave the numerical sort, every term, and numerical equality untouched. Both numerical composites therefore have the identity graph $x = y$; no quotient, retract, numeral restriction, or extra coding sort is involved. □

For the first-order bi-interpretability statement, use the ordinary realization that reads every canonical $\mathsf{D}_{x,y}B$ uniformly as $\exists x(B(x,0) \to \forall y B(x,y))$ and leaves numerical equality and all scalar terms unchanged. Composing the displayed translations and composites with the inherited full-domain bridge between $\mathcal{C}$ and iPRA yields a bi-interpretation of this ordinary realization of $\mathcal{G}$ with full first-order iPRA. The contradiction $0 = 1$ is unchanged, so equiconsistency follows as well. The inherited arithmetic bridge supplies the primitive-recursive compiler and the ordinary logical interpretation; this section does not claim those inherited constructions as new.

B.5 Why an unrestricted checker can still be sound

There is a useful, but strictly separate, administrative calculus $\mathcal{U}$. It permits arbitrary atomic paired matrices and uses the same root-determined DI, with ordinary Q-style clauses on DE. It is not claimed that all of $\mathcal{U}$ is bi-interpretable with iPRA.

Give arbitrary administrative formulas their following ordinary interpretations:

$$(s = t)^\dagger = (s = t),$$

$$(\mathsf{D}_{x,y}B)^\dagger = \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ is a literal equation,} \\ \exists x\, (B^\dagger(x,0) \to \forall y\, B^\dagger(x,y)), & B \text{ is compound.} \end{cases} \quad \text{(Mixed administrative reading)}$$

Retained clauses keep the ordinary reading $A^\dagger \to \forall y\, B^\dagger$. If the raw codec also permits administrative Q-closures as formula-shaped objects, including under transport or a paired binder, extend this interpretation by $(Q_y B)^\dagger = B^\dagger(0) \to \forall y B^\dagger(y)$. Such a matrix is compound for root-determined DI. This adds no canonical object-formula constructor and preserves the following soundness argument.

Every $\mathcal{U}$ inference is sound in iPRA under this reading. Atomic DI directly proves its progressive reading; compound case DI uses numerical coverage. In atomic DE, contextual equation induction turns $\forall y(E(y) \to E(Sy))$ into $E(0) \to \forall y E(y)$, exactly the clause that DE supplies. All other rules are immediate, including transport: numerical substitution does not change whether a matrix is an equation.

On canonical zero-initialized matrices, the mixed and ordinary readings agree provably in iPRA. Indeed $B(x,0)$ is a theorem, so, with x fixed,

$$\forall y\, (B(x,y) \to B(x,Sy)) \quad \longleftrightarrow \quad B(x,0) \to \forall y\, B(x,y).$$

The forward direction is equation induction; the reverse uses the true base to obtain every instance. Structural induction extends this agreement to all canonical guarded formulas.

Thus administrative proofs with unguarded intermediate formulas are conservative on canonical guarded conclusions: translate an accepted derivation into iPRA, use the canonical agreement, and translate back through the established bi-interpretation. Conversely, every canonical guarded derivation is already an administrative derivation.

This justifies a checker contract consisting of soundness and completeness for canonical guarded statements while allowing broader administrative certificates. It does *not* make the raw checker a validator of the guarded public grammar. It also does not assert completeness for every noncanonical encoding under the mixed reading. If a comparison requires rejection of all noncanonical public goals, the corresponding syntax validator must be implemented and counted.

B.6 Why merely deleting the choice would not suffice

Without the guarded canonical language, root-deterministic DI does not preserve the original native consequence relation. Let

$$p(y) = I(y, 0; u.C(u, 1, 0)), \qquad E(y) \equiv (p(y) = 1),$$

and leave the existential witness coordinate unused. Computation gives $p(0) = 0, p(1) = 1, p(2) = 0$. The original case introduction proves $\mathsf{D}_{x,y}E(y)$: its base gives $0 = 1$, from which equality transport and selector computation prove every equation.

By contrast, the progressive reading of this formula is false in the standard natural numbers, because $E(1)$ holds and $E(2)$ does not. The mixed interpretation proves that it is not a theorem of the unrestricted root-deterministic calculus. The guard, the explicit translations, and both composite proofs are therefore substantive parts of the reduction, rather than optional notation.

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [8] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [9]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary

primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[\left(E(0, \vec{p}) \wedge \forall n \big(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}) \big) \right) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [10]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n + 1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k + 1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$
$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u,\vec{p})$, introduce $i_h(n,a,\vec{p})$ with the two equations

$$i_h(0,a,\vec{p}) = a, \qquad i_h(Sn,a,\vec{p}) = h(i_h(n,a,\vec{p}),\vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

F.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem F.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

G Removing literal zero from the source of transport

The following is a restriction of a mathematical inference rule, rather than a change in the spelling of zero. Throughout this section, "nonzero source" means *not syntactically the single term* 0. Every variable and every term headed by S or I is a permitted source, including terms whose numerical value is zero.

Let $\mathcal{C}$ be the native scalar calculus with terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h),$$

the two computation schemas

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

formulas generated by equality and the Q, D binders, and the Hyp, QE, DE, case-DI and equation-progress-DI rules stated in the main inventory. The same argument applies to the original presentation with D object formulas and Q resources in contexts. Let $\mathcal{C}^-$ replace unrestricted transport by

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr}^-, \qquad s \not\equiv 0. \tag{Tr$^-$}$$

Here F is an arbitrary native numerical template, and $s \not\equiv 0$ is a condition on finite term syntax. All other rules are unchanged.

Theorem G.1 (Literal-zero source elimination)**.** *For every native context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathcal{C}^-} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}} A.$$

In particular, the ordinary whole-domain realization of $\mathcal{C}^-$ *is bi-interpretable and equiconsistent with full first-order iPRA through the same interpretations as the native scalar core. No change of numerical domain or interpreted equality is needed.*

We prove the theorem without assuming arbitrary numerical substitution admissibility for $\mathcal{C}^-$: such substitution can turn a variable source into the forbidden literal zero. Every occurrence of a helper equation at an arbitrary term is either obtained directly from its displayed derived rule at that term, or from a derivation whose transport sources retain an S or I head under the indicated substitution.

G.1 Structural operations, cases, and equation induction

Weakening, ordinary-assumption cut, and capture-free renaming are admissible by induction on derivations. These operations do not turn a nonzero source into literal zero. As usual, fresh parameters are renamed before a cut. We also use the following restricted substitution fact: a capture-free substitution is admissible for a particular finite derivation if it leaves every primitive transport source in that derivation syntactically different from zero. This fact follows by the same induction; it is not unrestricted substitution admissibility.

Lemma G.2 (Cases and equation induction need no transport)**.** *In $\mathcal{C}^-$ the following rules are derivable:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}\ \mathsf{NC}, \qquad \frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}\ \mathsf{EI}.$$

In NC, A is any native formula; in EI, E is one literal equation. The parameter z is fresh for Γ, t, and the fixed parameters of the matrix. Other parameters stay fixed throughout EI.

Proof. Choose an unused witness coordinate x. In NC, weaken the successor proof by $A(0)$ and apply case-DI, with witness zero, to obtain $D_{x,y}A(y)$. DE opens $Q_yA(y)$, because x is unused. QE with the given base proof and target t yields $A(t)$, and DE closes. For EI use progress-DI instead, with the given equation step, and use the same DE/QE construction. No transport rule occurs. □

These rules will be applied to unconditional helper equations before they are weakened into the context containing $0 = t$. We never use NC with a case parameter occurring in its retained context.

G.2 The equality operations available immediately

Lemma G.3 (Reflexivity and safe equality operations)**.** *Reflexivity $r = r$ is derivable for every term r. From $p = q$ with $p \not\equiv 0$, one can derive $q = p$ and $H(p) = H(q)$ for any numerical template H. Every direct computation can be used in either direction as an equation; using the reversed equation as a transport major is permitted only when its new source is syntactically nonzero.*

Proof. Put $b = I(0, r; u.0)$. The computation $b = r$, used as both major and minor of transport with template $v = r$, gives $r = r$. Its source is the nonempty iterator term b. For symmetry of $p = q$, transport the reflexivity $p = p$ through the template $v = p$. For numerical congruence, transport the reflexivity $H(p) = H(p)$ through $H(p) = H(v)$. Both use the permitted source p. Every computation equation has an I-headed left side, so its symmetry follows by the same construction. □

An occurrence-selective template may keep some occurrences of a term fixed while abstracting other occurrences. This is ordinary capture-free template substitution, not an extra inference rule.

Lemma G.4 (Joining two nonzero terms known to be zero)**.** *Suppose $p, q \not\equiv 0$ and Γ proves $p = 0$ and $q = 0$. Then $\Gamma \vdash p = q$ using only Tr^- and computation.*

Proof. Let

$$z_0 = I(0, 0; u.0), \qquad b_p = I(0, p; u.0).$$

Computation gives $b_p = p$. Start with $b_p = b_p$ and use the major $p = 0$ in the template $I(0, v; u.0) = b_p$. This gives $z_0 = b_p$. Symmetry at the nonzero source z_0 gives $b_p = z_0$. Now transport through $v = z_0$ using the computation $b_p = p$, whose source is b_p, to obtain $p = z_0$. The identical construction gives $q = z_0$. Symmetry at source q gives $z_0 = q$, and transport at source z_0 in $p = z_0$ gives $p = q$. The sources used are p, q, z_0, b_p, b_q; all are syntactically nonzero. No reverse transport from zero has been used. □

The zero-join lemma is deliberately placed before the next construction. The positive case of its singleton test needs precisely this lemma; ordinary transitivity through zero is not yet available.

G.3 A singleton-one test without predecessor

Define the finite scalar terms

$$\begin{aligned} C(v,a,b) &:= I(v,a;u.b), & Z(v) &:= C(v,1,0),\\ K(v) &:= C(v,0,1), & H(v) &:= C(v,1,2),\\ N(n) &:= I(n,0;u.Z(u)), & R(n) &:= I(n,0;u.H(u)),\\ L(n) &:= N(R(n)). \end{aligned} \tag{Tests}$$

The update variable in C is unused. Here $1 = S0$ and $2 = S1$. The standard-model description of R is $0, 1, 2, 2, \ldots$; the proof below establishes the needed equations internally and does not assume this description.

Lemma G.5 (Uniform test equations)**.** *The following equations have derivations in $\mathcal{C}^-$, uniformly in the displayed argument:*

$$H(v) = SK(v), \qquad R(1) = 1, \qquad R(SSv) = 2,$$

$$N(1) = 1, \quad N(2) = 0, \quad L(1) = 1, \quad L(SSv) = 0, \quad \boxed{L(Sv) = Z(v)}. \tag{Singleton}$$

All transport sources in these helper derivations have an I or S head, except for the explicit nonzero endpoints in the zero-join lemma; those endpoints also have an I head in this application.

Proof. For $H(v) = SK(v)$ use NC. At zero both sides reduce to 1; at a successor both reduce to 2. To join two equations $p = j$, $q = j$ with $j = 1$ or $j = 2$, reverse $q = j$ at its nonzero source q, then transport at the nonzero numeral j. Each reduction of these particular terms is a forward replacement of an iterator computation. This proves the helper at an arbitrary target v by NC, without substituting zero into a variable-source transport.

The equations $N(1) = 1$ and $N(2) = 0$ are finite computations: $N(0) = 0$, $N(1) = Z(N(0)) = 1$, and $N(2) = Z(N(1)) = 0$. Every replacement source in these calculations is an iterator term. Likewise $R(1) = H(R(0)) = H(0) = 1$.

Computation and the first helper give, for any v,

$$R(Sv) = H(R(v)) = SK(R(v)).$$

Use this equation at the nonzero source $R(Sv)$ in $R(SSv) = H(R(Sv))$. The resulting right side is $H(SK(R(v))) = 2$ by the successor computation of C. Thus $R(SSv) = 2$. These substitutions are all at I-headed sources. Substitute $R(1) = 1$ and $R(SSv) = 2$ inside N and use the two computed values of N to obtain $L(1) = 1$ and $L(SSv) = 0$.

Finally apply NC to the matrix $L(Sv) = Z(v)$. At zero, $L(1) = 1$ and $Z(0) = 1$ are joined through the nonzero numeral 1. At a successor, $L(SSv) = 0$ and $Z(Sv) = 0$ are joined by Lemma G.4; both left sides are nonempty iterator terms. This proves the boxed equation at every target. Its use is therefore an instance of a displayed derived rule, not an appeal to unrestricted substitution admissibility. □

Lemma G.6 (An unconditional zero-filter identity)**.** *For every term t,*

$$t = C(Z(t), t, 0) \tag{Filter}$$

is derivable in the empty context of $\mathcal{C}^-$.

Proof. Apply NC to $v = C(Z(v), v, 0)$, with the target t. At zero, forward computations give $C(Z(0), 0, 0) = C(1, 0, 0) = 0$; reverse this equation at its nonzero left side. At Sv, forward computations give $C(Z(Sv), Sv, 0) = C(0, Sv, 0) = Sv$; again reverse at the nonzero iterator left side. Thus the base and successor proofs have no retained assumption depending on the case variable. All numerical arguments of the final NC target are permitted. In particular, this identity is proved before any assumption $0 = t$ is used. □

G.4 Turning a zero-source equality around

Lemma G.7 (Zero-source symmetry)**.** *For every term t,*

$$0 = t \ \vdash_{\mathcal{C}^-} \ t = 0. \tag{ZS}$$

Proof. If $t \equiv 0$, use reflexivity. Otherwise t is a permitted source. Work in the fixed context $\Gamma = \{0 = t\}$ and put

$$A(n) := I(n, t; u.Su).$$

We first prove $n = A(n)$ by EI, keeping t fixed.

For the base, computation gives $A(0) = t$. Its symmetry gives $t = A(0)$. Since $t \not\equiv 0$, transport at source t in the retained assumption $0 = t$ gives $0 = A(0)$.

For the step, retain $z = A(z)$ with z fresh. This source is a variable, hence is permitted. Numerical successor congruence from Lemma G.3 gives $Sz = SA(z)$. Computation gives $A(Sz) = SA(z)$, and its symmetry gives $SA(z) = A(Sz)$. Its source $SA(z)$ is also permitted, so transport in $Sz = SA(z)$ gives $Sz = A(Sz)$. This is exactly the EI step. The induction hypothesis is used at the same fixed parameter t.

At target 1, EI gives $1 = A(1)$. Forward computation gives $A(1) = St$: the successor equation produces $SA(0)$, and the nonzero-source computation $A(0) = t$ is used inside S. Transport at source $A(1)$ consequently gives

$$1 = St.$$

Its source 1 is permitted. Numerical congruence for L gives $L(1) = L(St)$. Use $L(1) = 1$ at source $L(1)$, and $L(St) = Z(t)$ at source $L(St)$, obtaining $1 = Z(t)$. Symmetry at source 1 gives $Z(t) = 1$.

Weaken the unconditional Filter identity into Γ. Transport at the iterator source $Z(t)$ changes its right side to $C(1, t, 0)$. The latter computes to zero, and its source is again an iterator term. The resulting equation is $t = 0$, as required. □

This proof uses cases only for unconditional helper equations. The sole induction with the retained assumption $0 = t$ has the literal equation $n = A(n)$ as invariant. It does not perform a forbidden case distinction while silently changing the retained context.

G.5 Recovering equality algebra and formula transport

Lemma G.8 (Full numerical equality algebra)**.** *The calculus $\mathcal{C}^-$ admits unrestricted symmetry, numerical congruence, and transitivity. In particular it admits root replacement*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash t = r}.$$

Proof. Symmetry uses Lemma G.3 when the source is not zero and Lemma G.7 otherwise.

For numerical congruence from $s = t$, a nonzero s is handled by Lemma G.3. If $s \equiv t \equiv 0$, use reflexivity. If $s \equiv 0$ and $t \not\equiv 0$, first derive $t = 0$, apply permitted numerical congruence at source t to obtain $H(t) = H(0)$, and use the now established full symmetry.

For transitivity from $r = s$ and $s = t$, a nonzero middle term s is handled by one permitted transport. If $s \equiv 0$ and either endpoint is literally zero, a given premise is already the conclusion. Otherwise $r, t \not\equiv 0$; reverse $0 = t$ to get $t = 0$ and apply the zero-join lemma to $r = 0$ and $t = 0$. Root replacement follows by symmetry and transitivity. These constructions are acyclic: zero-source symmetry was proved before any unrestricted numerical congruence or transitivity. □

Lemma G.9 (Full transport)**.** *For every native numerical template $F(v)$, the rule*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}$$

is admissible in $\mathcal{C}^-$, without a source restriction.

Proof. Use external induction on the finite number of formula binders in F, uniformly in retained contexts and fixed numerical parameters. For an equation $F(v) \equiv A(v) = B(v)$, use unrestricted numerical congruence to obtain $A(s) = A(t)$ and $B(s) = B(t)$. Join these with $A(s) = B(s)$ using the established equality algebra.

For $F(v) = D_{x,y}B(v, x, y)$, eliminate $F(s)$ at a fresh a, retaining $Q_yB(s, a, y)$. Introduce the desired $F(t)$ by case-DI with witness a. Its local assumption is $B(t, a, 0)$. Full symmetry gives $t = s$, and the external induction hypothesis for the smaller matrix transports the assumption to $B(s, a, 0)$. QE gives $B(s, a, Sz)$, and the induction hypothesis in the forward direction gives $B(t, a, Sz)$. Close case-DI and DE. All new parameters can be chosen fresh for the source equality and the retained context.

If Q is first-class formula syntax, first observe that its case introduction is derived without transport: introduce $D_{u,y}B(y)$ with unused u by case-DI and eliminate it with the one-node continuation $Q_yB(y)$. For $F(v) = Q_yB(v, y)$, use this derived introduction for the target. Convert its base assumption backwards by the induction hypothesis, apply the old Q resource, and convert the successor conclusion forwards. This is the same smaller-matrix argument and uses no new numerical induction. □

Proof of Theorem G.1. Every restricted derivation is an unrestricted one. Conversely, replace each unrestricted transport node by the derivation supplied by Lemma G.9, recursively translating its premises first. Other inference nodes are unchanged. The result has the same context and conclusion. Thus the full native consequence relation, not just closed theorems, is preserved. The existing native/ordinary and ordinary/iPRA interpretations and both identity composites therefore transfer unchanged. □

G.6 Compatibility with restricted computation certificates

Some compact checkers require the initializer of a computation packet to have nonempty syntax, and encode a successor counter by a nonempty predecessor. These are certificate restrictions, not restrictions on the two mathematical computation schemas. They do not introduce a circular dependence on zero-source transport in the preceding proof.

Use the inherited certified-zero term

$$c_0 = I(2, 1; u.0).$$

Its equation $c_0 = 0$ is a directly admitted successor computation: the predecessor and initializer are both 1. For an arbitrary computation instance with literal-zero initializer or literal-zero successor predecessor, form its whole equation as a template $J(v)$, replacing just these offending occurrences by v. The instance $J(c_0)$ is a directly admitted computation. Transport through this equation template using $c_0 = 0$ yields $J(0)$, the desired original instance. Both computation premises are admitted directly, and the only transport source is the nonempty term c_0.

This is exactly the inherited three-node normalization in `Research/certified_zero/tests/producer.py`, function `expand_computation`: one transport and two computation premises. Its source is c_0, even when both defects are corrected simultaneously. Passive zero occurrences in an update are left alone. Thus all computation instances used in the mathematical proof can be expanded before compiling to the source-restricted checker.

G.7 Scope of the result

The new statement is the admissibility of all omitted literal-zero source transports. The native term/formula language, arithmetic schemas, and target theory remain the inherited ones. This theorem justifies replacing a two-way equality/Q major-pattern test by a source-discriminated test in a suitable certificate protocol. It does not assert a minimum proof-checker size, a shortest proof expansion, or a new characterization of the primitive recursive functions. The displayed proof is a written universal derivation; executable certificates, when supplied, validate particular instances rather than substitute for this argument.

H A uniform arity encoding and its checking theorem

The arithmetic and object logic remain the scalar-iteration Eq/D calculus specified above. The change is in its finite certificate representation. A paired formula is unary, an equation is binary, and an internal closure is ternary:

$$\mathsf{D}_{x,y}B \longleftrightarrow [B], \qquad s = t \longleftrightarrow [s,t], \qquad \mathsf{Q}_y B \longleftrightarrow [B,B,B]. \tag{Arity encoding}$$

The three copies in a closure are representation redundancy: only the third has mathematical content. Their benefit is that tripling a unary major constructs exactly its closure. Tripling any other nonempty formula array produces an array of arity at least six, which will denote truth. This makes the other major cases neutral without a separate test.

The previous generated-assumption and constructed-computation interfaces are inherited from the corpus [6, 5]. The new encoding makes a single arithmetic expression, $5 - \text{arity}$, perform the necessary depth adjustments. Its soundness depends on an explicit interpretation of surplus syntax. It would not be sound to drop arity checks while continuing to treat every surplus array as an unconstrained new connective.

H.1 The complete sorted interpretation of arrays

Write $\top$ for the fixed equation $0 = 0$. A protected numerical slot has width two. The numerical root has depth three: the coordinates $0, 1, 2$ all denote zero, and under p protected slots the depth is $3 + 2p$. Coordinates $3 + 2j$ and $4 + 2j$ name slot j, counted from the innermost slot, for $0 \le j < p$. Formula depth is $2p$ under the same p slots. Negative coordinates admitted by the current global floor name rigid numerical parameters.

The decoder is sorted by the position in which a tree is used. Let ρ be its ordered environment of protected numerical slots. The notation (a, ρ) means prepend a slot of value a. A superscript $\sharp$ denotes a child decoded in the indicated environment.

Numerical arity	Meaning	Relevant child environment
0	0	No child
1: $[b]$	$Sb^\sharp$	$(0, 0, \rho)$
2	0	Annotations ignored
3: $[n, a, h]$	$I(n^\sharp, a^\sharp; v.h^\sharp)$	$(0, \rho)$ for n, a; (v, ρ) for h
≥ 4	0	All children ignored

Formula arity	Meaning	Relevant child environment
0	$\top$	No child
1: $[B]$	$\mathsf{D}_{x,y}B^\sharp$	(y, x, ρ)
2: $[s, t]$	$s^\sharp = t^\sharp$	ρ numerically
3: $[P, U, B]$	$\mathsf{Q}_y B^\sharp$	(y, ρ) for B
≥ 4	$\top$	All children ignored

The first two entries of a ternary formula are annotations. A canonical closure copies its body into both of them, as in (Arity encoding). Successfully admitted scalar formula padding also denotes $\top$. An internal closure has the ordinary meaning $B(0) \to \forall y B(y)$; it is not an equation and is not eligible for compound induction.

Canonical encoded object expressions use arrays and integer coordinates, respect these slots, and contain no high-arity node. The checker itself still generates high-arity truth entries in administrative contexts, even during a canonical proof. In particular, constructing a successor beneath positive bound coordinates requires passing through two dummy zero slots. Constructing an iterator requires one new slot: zero in its counter and seed, and its genuine state variable in the update. Rigid negative parameter names are unaffected by these shifts.

The additional raw shapes are certificate aliases with the displayed meanings. They are not independent predicate or function symbols added to the object theory. Our exact completeness statement concerns the canonical native language. We do not require every possible raw spelling of a semantic alias to be interchangeable under the same raw certificate.

H.2 One depth calculation replaces the lookup table

For every array the opener recursively adds $5 - \text{arity}$ to its depth. The meaningful cases are

arity	1	2	3
depth increment	4	3	2.

Unary D adds two numerical binders; unary successor adds two dummy zero slots. Binary equality changes formula depth $2p$ to numerical depth $3 + 2p$. Ternary Q adds one numerical binder; ternary iteration adds one state slot, or one dummy slot in its counter and initializer. The same calculation therefore covers all relevant constructors.

Binary numerical annotations denote zero, so their parity-changing traversal has no mathematical effect. Empty arrays have no children. At arity at least four, the entire node already denotes a constant. Even a zero or negative depth increment below such a node cannot expose one of its ignored children as a meaningful variable. Traversal may still reject an alias; it cannot change its stipulated interpretation.

This is a semantic assertion, not raw idempotence of the opener. For example, at floor -1, opening a six-element array of zeros with image -1 can yield six copies of -1, while a later no-image opening of that result fails at a negative child depth. Both formula arrays denote $\top$. The proof below only concerns successful calls at meaningful sorted positions. Canonical input object expressions contain no surplus arrays; the administrative exports that generate them are justified separately as truth entries.

H.3 Opening and scope

Write $i(a, t, d)$ for the executable opener at floor k. It preserves a scalar when $k \leq a < d$. At a numerical position, every other relevant scalar is the distinguished substitution hole; the supplied image is checked at numerical root depth three before insertion. At a formula position scalar padding is truth and carries no numerical hole. Missing images at used numerical holes reject.

Lemma H.1 (Semantic opening)**.** *For every successful call at a meaningful formula root of depth $2p$ or numerical root of depth $3 + 2p$, with $p \geq 0$, decoding its result is capture-free filling of the distinguished hole in the decoded input template. These depths protect p numerical slots; admitted rigid parameters are preserved. If an image is semantically unused, zero may be chosen as its mathematical representative. Recursive calls at negative depths inside ignored annotations need no independent sorted interpretation.*

Proof. Induct on the finite raw tree. At a relevant scalar, the interval comparison is exactly the protected-coordinate test. At a meaningful unary, binary, or ternary node, the preceding tables supply precisely the required additional slots. At numerical binary annotations, high-arity nodes, and ignored closure fields, the relevant mathematical meaning is constant or absent, so the conclusion is immediate regardless of traversal inside the annotation.

An inserted image is checked at its own numerical root. Its free parameters are rigid negative names, and it has no unbound positive coordinate there. Insertion beneath protected slots therefore captures none of its parameters; its internal binders retain their relative indices. This proves the inductive step for substitution as well as for mere traversal. □

A successful no-image check consequently preserves the interpretation of its checked closed expression. Raw structural equality of checked outputs implies equality of their decoded meanings. The checker does not infer equality merely from a semantic comparison of two different programs or numerical terms.

The global floor decreases by one at each non-hypothesis continuation. If its old value is K, the integer $K - 1$ is the designated fresh eigenparameter. With integer coordinates it is the only new name. If fractional coordinates are accepted, other names in $[K - 1, K)$ may also occur in the finite certificate. These are additional fresh parameters. They must not be silently equated to the designated integer eigenparameter.

Previously checked relevant goals and retained hypotheses mention none of these new names: their earlier no-image checks would reject a relevant numerical occurrence below the old floor. Coordinates inside ignored annotations do not contribute free variables to the decoded formula. This is the freshness invariant used in every rule below.

H.4 A computation hypothesis only needs a zero instance

The shared continuation can open a generated arithmetic equation at a fresh parameter. That equation need not be a theorem uniformly in the new parameter. The following admissible rule is sufficient.

Lemma H.2 (Supply with a known zero instance)**.** *Suppose $\vec{a}$ is fresh for Γ, F, the calculus proves $H(\vec{0})$, and $\Gamma, H(\vec{a}) \vdash F$. Then $\Gamma \vdash F$.*

Proof. Substitute zero for every member of $\vec{a}$ in the second derivation. Freshness leaves Γ, F unchanged. Cut in the derivation of $H(\vec{0})$. Both substitution and cut are admissible in the native calculus. This is a finite proof transformation and does not use a consistency assumption. □

All names newly admitted when the floor changes belong to $\vec{a}$; for noninteger aliases this includes the finite collection in $[K - 1, K)$ that actually occurs. On decoding, their simultaneous zero substitution agrees with opening the generated expression at zero using the old floor. Previously admitted parameters are preserved. New rigid names and unprotected placeholders both become zero. The opener may keep different raw zero spellings, but their decoded values agree.

The constructed-computation packet supplies a triple $s = [n, b, r]$ and a mode u. In successor mode its generated equation has raw form

$$\big[\,[[n], b, r],\ i(r, s, 3)\,\big], \qquad \text{(Step supply)}$$

while in zero mode it has form

$$[\,[u, b, r],\ b\,]. \qquad \text{(Zero supply)}$$

Here successor mode means that u is truthy; the zero mode has falsy u. These expressions describe actual arrays, before their final opening. In (Step supply) the counter is constructed as a successor, rather than accepted on an unsupported assertion that it is one. This construction is inherited from the corresponding draft interface [5].

Lemma H.3 (Sound constructed computation)**.** *Every successful arithmetic supply has a decoded zero instance derivable from the two scalar-iteration computation schemas.*

Proof. First apply the simultaneous zero substitution described above. In zero mode every successful falsy numerical counter denotes zero, so (Zero supply) is an $I0$ instance. In the exact-list profile the relevant accepted aliases are an empty array, numeric zero, false, and numerically equal zero values. Null, strings, and dictionaries cannot enter a meaningful numerical position through an unchecked coercion.

In successor mode the left counter is an actual successor. If the update uses its state in a semantically relevant position, opening it invokes the no-image check of the recursive source s at numerical root three. This checks the entire source, including its update. Only coordinates three and four are aliases for its single state slot. A genuinely escaping coordinate in that update is revisited during the recursive-image check and rejects. Its counter and initializer are likewise closed with their prescribed dummy zero slot.

Inserting the extra dummy slots of the displayed successor does not alter such a checked predecessor: extra unbound slots have value zero, and actual free parameters are rigid. Lemma H.1 therefore identifies the right side of (Step supply) with the update applied to exactly the recursive predecessor value occurring on its left. This is the IS schema.

If the update has no semantically relevant state occurrence, its decoded value is constant in that variable. Its zero-normalized value is the same in the source update and on the right, independently of the unused recursive image; this is again an IS instance. Traversal inside an ignored annotation may still inspect the image and reject. That removes a certificate and does not invalidate the argument for a successful one.

Finally, successful three-field source extraction cannot hide another JSON operator. A scalar is not iterable. A three-character string, or a three-key JSON dictionary, necessarily puts a nonempty string in a relevant source field, which the inspected opener rejects. The successful source is therefore the stated array triple. □

The recursive-image check in the state-dependent case is essential. Merely opening a final displayed equation at zero would not establish that its two state interpretations agree. Combining Lemma H.3 with Lemma H.2 justifies both actual image choices of the common continuation, including the branch that retains a fresh-parameter hypothesis.

H.5 Audit of the logical continuations

Hypothesis closure. The initial opener check and membership comparison select the same decoded context formula. Additional aliases can make a spelling less convenient, but cannot identify an equation with a false denotation solely by semantic evaluation.

Equality transport. The major must match $[s, u]$, and opening the proposed formula template at u must match the target. The continuation checks its opening at s. Lemma H.1 gives ordinary transport along the decoded equality. The argument applies to compound formulas, internal clauses, and truth aliases as well as equations.

Closure specialization. The closure pattern is the triplicate $[t, t, t]$. The guard forces the old instance to have numerical image zero; the new instance may have an arbitrary numerical image. This is exactly $B(0) \to \forall y B(y)$. The zero guard is indispensable. Without it, a closure with false zero instance and true successor instances would turn a proved successor instance into falsity.

Paired elimination. The export is always three concatenated copies of the major. A unary major $[B]$ therefore exports $[B, B, B]$. All three copies are opened identically; the third copy has precisely the witness instance required by paired elimination. The newly allocated witness is fresh for the retained context and conclusion.

An equation major instead exports arity six; a closure major exports arity nine; a larger array exports still larger arity. These are all truth aliases. Thus failed elimination endpoint claims have only the effect of adjoining a theorem. A numeric scalar major, if accepted as formula padding, exports scalar padding after multiplication and also denotes truth. Unsupported types reject.

A ternary formula with unequal annotations still denotes the closure of its third child. It fails the triplicate specialization pattern unless the three fields match. This loses a possible raw spelling, not a mathematical soundness condition. Canonical compilation uses triplicate closures throughout.

Paired introduction. The call `i(*a,t,2)` uses the function's argument count to enforce the relevant outer arity. For a unary goal $a = [B]$, it is exactly `i(B,t,2)`: the universal slot is protected while the existential witness is opened. For arity at least two, there are too many arguments and the call rejects. An empty goal can instead make the legal two-argument call `i(t,2)`. Its original target was already $\top$, so accepting it cannot add a false theorem.

For the unary goal, the subsequent successor opening gives the intended premise $B(t, Sz)$. The local hypothesis is $B(t, 0)$ in case mode and $B(t, z)$ in progress mode. The freshness invariant supplies the usual eigenvariable conditions.

Induction remains restricted. Among meaningful nonconstant matrix constructors, only binary equality has even arity. Unary paired formulas and ternary closures have odd arity and receive case introduction. Empty or surplus arrays that reach the progress branch denote the fixed equation $0 = 0$, whose induction is valid. Scalar padding that cannot provide a length rejects. The broader certificate alphabet therefore adds no induction on a quantified matrix.

Additional fresh parameters. An accepted subderivation can contain newly admitted fractional names, but the older context and target do not depend on them. Replace these auxiliary names by zero when closing the rule, retaining its designated integer eigenvariable where that rule needs one. The common opening of an already checked retained goal leaves its decoded meaning unchanged. This handles the actual global-floor convention without pretending that every admissible negative number is an integer.

H.6 Canonical completeness, including zero witnesses

Rename a finite native proof's eigenvariables to fresh negative integers. Encode its terms with the slot conventions of Section H.1; encode paired formulas as unary and closures as triplicate. Hypothesis leaves consume no instruction. Transport, closure specialization, and paired elimination have their displayed packets. Arithmetic computation leaves use (Zero supply) or (Step supply) and then close the target by hypothesis lookup.

One representational issue requires an actual proof transformation. The packet's truthy witness field selects introduction. A literal zero witness therefore cannot simply be inserted into that field.

Define

$$J(t) = I(0, t; v.0).$$

The computation axiom proves $J(t) = t$, and $J(t)$ has a nonempty array representation even when its value is zero. To introduce with a falsy witness spelling t, introduce instead with $J(t)$: transport its retained assumption to the assumption using t, use the original premise derivation, and transport the resulting successor instance back to the instance using $J(t)$. Derived symmetry supplies the reverse transport. This is a finite transformation of the premise, and the paired conclusion itself is unchanged. If the existential coordinate is unused, any closed truthy witness works without a conversion.

The scalar coordinate 1 is also a semantic zero at numerical root three, but that observation alone is not a completeness proof. Closure specialization and case introduction insert literal raw zero, and distinct zero aliases need not be interchangeable under a fixed raw proof spelling. The computation-backed box gives the required derivation instead of assuming representational interchangeability.

The mode zero selects progress at an equation. A different falsy mode, such as the empty array, selects case. These are certificate control tags, not numerical assertions about the witness. All steps in the compilation are finite and effective.

Theorem H.4 (Conservativity and completeness of the arity checker)**.** *Every accepted certificate has its decoded conclusion provable in the ordinary realization of the native calculus. For every canonical native end sequent, the internal checking judgment and the native calculus have the same consequence relation, in the idealized unbounded integer-and-array implementation. In particular, the public empty-context driver recognizes exactly the canonical closed theorems. The translation in each direction is effective on finite derivations.*

Proof. For soundness, induct on the finite accepting call tree. Hypothesis closure uses Lemma H.1; the logical cases are the audited continuations above. Arithmetic supply uses Lemmas H.3 and H.2. Every discarded annotation is accounted for by its zero or truth interpretation, and every fresh parameter by the scope invariant. For completeness, compile a finite native derivation by the preceding construction, including the boxed-witness transformation. Its canonical input expressions require no surplus-arity aliases or fractional names; generated administrative surplus arrays have exactly the truth meaning already proved. This proves both implications without using a finite test suite as a substitute for induction on derivations. □

The unchanged native calculus is bi-interpretable with full first-order iPRA by the inherited arithmetic and logical bridge [6, 4]. In particular,

$$(\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)).$$

The whole numerical domain and actual equality are retained. Its internally proved numerical round trips and its formula-composite derivations therefore transfer through Theorem H.4. We have changed the checked representation, not replaced bi-interpretability by standard-model agreement or supplied a new proof of the scalar predecessor theorem.

H.7 JSON and finite-runtime qualifications

The primary profile uses an ordinary list-type test. The smaller byte profile instead tests multiplication by zero against an empty list. Among standard JSON runtime values the latter recognizes exactly lists: numeric values and booleans produce numeric zero, strings produce an empty string, and null and dictionaries raise rather than becoming arrays. This argument assumes ordinary decoded JSON values, not arbitrary Python objects with overloaded methods.

Finite nonnegative real coordinates have the same interval interpretation: $[0, 3)$ is zero, and $[3 + 2j, 5 + 2j)$ names slot j. Finite negative coordinates in the admitted range are additional rigid names. Equal Python numerical values have the same decoded role, including Boolean aliases and signed zero. Nonfinite values admitted by Python's permissive JSON reader cannot survive a relevant closed numerical check. They may be filled as template holes or ignored inside a constant annotation; neither makes them unchecked parameters.

The alternative opener based on generic iteration has a separate runtime qualification. Empty strings and empty dictionaries traverse as empty arrays; a dictionary whose sole key is empty traverses as a unary array with an empty child. Dictionary values are not visited. Nonempty strings can rely on CPython's recursion limit to reject, as also occurs in a current corpus comparison profile [7]. The idealized structural-termination claim for the exact-list profile must not be transferred to that generic profile unchanged.

All executable profiles have ordinary finite memory, stack, and input limits. These can reject a sufficiently large otherwise valid proof. The formal consequence relation is the unbounded finite-certificate calculus; the delivered program checks certificates within its runtime resources. The measurements charge the entire delivered checker, while the semantic proof and canonical producer are supplied separately.

I Complete selected source listings

The native module below includes every acceptance-critical component. The compact module has the separate contract explained in Section 4. Long printed lines are wrapped for typography; the counted files contain the normalized source exactly.

I.1 Fully validated native checker

```
import json

def c(a, g=[0], k=0, v=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 and (not v or len(a)
    in (1, 2 + d % 2)) else a if k <= a < d and (not v or (d % 2 and (not a or a > 1 == a % 2)))
    else (d % 2 or v) and i(t, d=3)
    if i(a) in g:
        return
    v = 0
    m, s, u, t = q()
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a :=
    i(*a, t, 2))
    c(i((a, i(t, s))[(m, a) == (s and [s, u] or [t] * 3, i(t, u))], [(k := (~-k))]), g + [i(h, (m or
     len(a) % 2 == u) and k)], k)
p = json.load(open(0))
q = not str(p).strip('0123456789-[], ') and p.pop
c(q(), v=1)
print('ACCEPT')
```

I.2 Compact guarded checker

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d
    else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a :=
    i(*a, t, 2))
    c(i((a, i(t, s))[(m, a) == (s and [s, u] or [t] * 3, i(t, u))], [(k := (~-k))]), g + [i(h, (m or
     3 % len(a)) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt repository, pinned commit `6ffc49f55d355b548f7a8e1530570aee5de086b3`. `https://github.com/VladimirReshetnikov/ProveIt`. Main project path: `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.

[2] ProveIt, `Research/validated_presentation/code/native_typed_eqd.py` and `Research/deterministic_introduction/implementation/`. Exact source identities and contract comparison are recorded in the delivered ledgers.

[3] ProveIt, incoming `ipra_smaller_envelope.zip`, `ipra_frontier_review_2026-11-12.zip`, and `ipra_orbit_closure.zip`, at the pinned commit. The first supplies the current 253-token and 275-AST coordinates; the last supplies the independently rerun corpus audit.

[4] ProveIt, the ninety `ConciseIPRA/Research/` packages, at the pinned commit. This is the comparison collection, not a universal lower-bound theorem.

[5] ProveIt, `Research/constructed_computation/`. Constructed scalar computation sources and their inherited supply interface.

[6] ProveIt, the generated-hypothesis and neutral-premise interfaces preserved in the `Research/certified_zero/` reference fragments and subsequent compact-checker packages.

[7] ProveIt, `Research/transient_seed/`. Historical recursor and decoder comparison referenced by the preserved arity-checker fragment.

[8] ProveIt, inherited scalar predecessor proof, `Research/certified_zero/reference/scalar_predecessor.tex`. Reproduced in Appendix C.

[9] ProveIt, inherited equality-reflection proof, `Research/certified_zero/reference/equality_reflection.tex`. Reproduced in this report's equality-reflection appendix.

[10] ProveIt, inherited pairing and primitive-recursion proof, `Research/certified_zero/reference/primitive_recursion.tex`. Reproduced in Appendix E.